SuzzaEngine

Source Code Reading Guide

- understanding a small Vulkan game engine from scratch -

Audience	Beginners with no prior graphics or engine experience
Stack	C++20 + Vulkan + GLFW + ImGui + glm
Code base	About 70 source files, roughly 10k lines
What this is	A reading map and key-mechanism notes, not an API manual
Code covered	main branch of this repository

2026-10

Reading Guide (Start Here)

This document solves exactly one problem: **in what order should a complete beginner read these ten thousand lines of code?**

It does not try to cover every class. It builds the skeleton first: what the project does, how it is compiled, how the code is layered, where execution starts, and how a single frame reaches the screen. Once the skeleton is clear, the details can be looked up directory by directory.

> **How to use this document.** Suggested use: on the first pass read only chapters 1 to 4 (about 40% of the text) and follow the seven-stop route in chapter 4 with the code open beside you. Treat chapter 5 as a reference manual and come back to it when you get stuck. Do the exercises in chapter 7 - reading without hands-on practice is forgotten within two weeks.

Structure of the document:

Chapter 1	Project overview: what it is, what it does, where its limits are
Chapter 2	Getting it to run: environment, build, three automatic steps
Chapter 3	Global structure map: directory layers and module duties
Chapter 4	Reading route: seven stops, in order
Chapter 5	Core mechanisms: lifetime, object model, rendering, reflection
Chapter 6	The full life cycle of one frame
Chapter 7	Hands-on exercises: from changing one line to adding a component
Chapter 8	Common pitfalls and troubleshooting
Chapter 9	Glossary
Annex A	File quick reference: one sentence per file
Annex B	Key API quick reference

Chapter 1 Project Overview

1.1 In one sentence

Suzza is a small 3D rendering engine written from scratch in C++20 and Vulkan, with a runtime ImGui editor, that organises scenes with the Actor-Component pattern.

It is a teaching / experimental engine: the code base is small, free of legacy baggage, and every module can be read end to end. That makes it a good place to understand how a game engine is taken apart and put back together. It is not a foundation for a commercial project - there is no physics, no animation system, no UI layout system, no hot reloading of assets and no multithreaded rendering.

1.2 The four core parts

Core part	What it does	Why a beginner should care
Actor-Component architecture	An Actor is a scene entity, a Component is a pluggable capability (mesh, camera, movement). Actors nest to form a transform hierarchy	This is the engine's world view. Understand it first and everything else becomes readable
Hand-rolled lightweight reflection	clang annotations plus an external LLVM tool generate class metadata, so the editor can list and edit fields by name at runtime	The most magical part of the project, and where beginners get stuck most often
Vulkan rendering back end	Hand-written instance, device, swapchain, render pass, pipeline, descriptors, command buffers, synchronisation objects	Largest amount of code, but clearly structured: can be read alongside a Vulkan tutorial
Runtime editor	ImGui panels: a hierarchy tree plus an inspector property table that edits scene objects directly	The best entry point for seeing why reflection is worth the trouble

Table 1-1 The four core modules

1.3 Stack and dependencies

All third-party libraries live under `thirdparty/` and have been trimmed. No package manager is needed:

Dependency	Purpose	Notes
Vulkan SDK	Graphics API and the `glslc` shader compiler	Must be installed, and `find_package(Vulkan)` must succeed
GLFW	Window, input, surface creation	thirdparty/glfw
glm	Vector, matrix and quaternion maths	thirdparty/glm
ImGui (docking branch)	Editor UI	thirdparty/imgui-docking
cgltf	glTF model parsing	thirdparty/cgltf, wrapped in tool/modelloader
stb_image	Image decoding (jpg/png etc.)	thirdparty/stb_image, wrapped in tool/imageloader

Table 1-2 Third-party dependencies

1.4 Code size

Everything written in-house under `src/`, `shared/` and `tool/` adds up to roughly 70 source files and about ten thousand lines. At 300 to 500 lines a day that is two to three weeks for a full pass. Only about fifteen files really deserve careful reading, and chapter 4 names each of them.

1.5 Scope: what it deliberately does not do

- No physics engine, no skeletal animation, no particle system.
- No scene serialisation: levels cannot be saved. The scene is built in code inside `SuzzaApplication::Init`.
- Lighting is minimal: no shadows, no post-processing, no deferred rendering.
- A material is just one texture plus one bindless slot, with no PBR parameter set.
- No asynchronous asset loading: models and textures are read synchronously on the main thread.

Knowing the limits matters. When you read the code and wonder 'why is X missing here', the answer is usually that X was never implemented, not that you failed to find it.

Chapter 2 Getting the Project to Run

Make it run before you read it. An environment where you can compile, see a picture and change one line to see the effect is worth three days of reading.

2.1 Environment

Item	Requirement
Operating system	Windows (the project is only verified with MSVC; Linux/macOS presets exist in CMake but are unverified)
Compiler	Visual Studio 2019/2022 with C++20 support
Build tools	CMake 3.21 or newer (the preset file uses preset v3) and Ninja
Graphics SDK	Vulkan SDK including glslc, version 1.3.x recommended
Other	Git

Table 2-1 Environment requirements

2.2 Three-step build

Open the **x64 Native Tools Command Prompt for VS** (important: only that shell has `cl.exe` and Ninja on its path), then change into the repository root:

```
cmake --preset x64-debug
cmake --build --preset x64-debug

# Output: out/build/x64-debug/SuzzaEngine.exe
# Just run it
```

For a release build, replace `x64-debug` with `x64-release`.

> **The most important build constraint. Do not use `cmake -G "Visual Studio 17 2022"`.** The reflection generator reads `build/compile_commands.json`, and only generators such as Ninja or Makefiles produce that file (see `CMAKE_EXPORT_COMPILE_COMMANDS ON` at `CMakeLists.txt:15`). Configure with the Visual Studio generator and the reflection step fails for lack of input, or silently keeps using stale generated code.

2.3 Three things the build does automatically

Understanding these three steps removes most of the 'inexplicable' build errors.

Step	Location in CMakeLists.txt	Output / effect
Compile shaders	lines 67-109	Runs `Vulkan::glslc` to turn `shaders/*.vert|frag` into `<build>/shaders/*.spv`
Generate reflection code	lines 112-135	Runs the generator executable under `tool/ReflectGenerated/`, scans `src/` and emits `*.generated.cpp`

Step	Location in CMakeLists.txt	Output / effect
Copy assets	lines 207-211	POST_BUILD copies `assets/` next to the executable

Table 2-2 The three automatic build steps

Asset paths are not guessed from the current working directory. `GetAssetsFolder()` and `GetShadersFolder()` in `shared/file/FileSystem.cpp` resolve them: first from the macros injected by CMake (`ASSETS_ROOT` / `SHADERS_ROOT`), otherwise from `assets/` and `shaders/` **next to the executable**. So copying assets to the wrong place, or moving the executable away on its own, shows up as 'failed to load model'.

2.4 Checking that it works

A successful start shows a 1600x1200 window titled `First Window`, three coloured cubes linked in a hierarchy, the ImGui hierarchy and inspector panels, left-drag to orbit the view and WASD to move (provided by `MovementComponent`).

If the window appears but the image is black, read the console log first (`SUZZA_LOG_ERROR` prints there). The most common cause is a Vulkan validation-layer report about descriptors or pipelines.

Chapter 3 Global Structure Map

3.1 The directory at a glance

```
SuzzaEngine/
|- CMakeLists.txt          top-level build: shaders, reflection, asset copy
|- CMakePresets.json       x64-debug / x64-release presets (Ninja + cl.exe)
|- shaders/                2 GLSL files, compiled to spv at build time
|- assets/                 models (gltf/bin) and textures
|
|- src/                    * engine and sample game
|  |- main.cpp             entry point, about 37 lines
|  |- SuzzaApplication.*   * sample 'game layer': builds the scene
|  |- engine/              * engine core
|  |  |- Engine.*          singleton: Init / Run / Destroy, owns managers
|  |  |- MainWindow.*      GLFW window, input, resize callback
|  |  |- BaseObject.h      base class of all resources (Destroy/IsAlive)
|  |  |- input/            InputManager: key and mouse state arrays
|  |  |- editor/           EditorUI: hierarchy + inspector
|  |  |- graphics/         Texture: VkImage + ImageView + memory
|  |  |- model/            Mesh / Material / Render / resource managers
|  |  '- vulkaninitialize/ one class per Vulkan concept + resource manager
|  |- scene/               * object model (ECS style)
|  |  |- Scene.*           actor registry, update, deferred destroy, loading
|  |  |- Actor.*           entity: transform, components, parent/child tree
|  |  |- Component.*       component base class + type id mechanism
|  |  |- components/       Mesh / Camera / Movement
|  |  '- actor/            PlayerController (an Actor subclass)
|  |- reflection/          runtime: ClassInfo / FieldInfo / ReflectionSystem
|  |- surface/             UI abstraction IUIRender + ImGui implementation
|  '- core/                logging, maths and other small utilities
|
|- shared/                 shared types, file system, macros, containers
|- tool/
|  |- broadcast/           hand-written unicast/multicast delegates
|  |- imageloader/         stb_image wrapper (manager + scheduler)
|  |- modelloader/         cgltf wrapper
|  '- ReflectGenerated/    * reflection generator exe + generated code
'- thirdparty/             imgui-docking / glfw / glm / cgltf / stb_image
```

Figure 3-1 Directory structure (* marks the parts worth close reading)

3.2 The layering view

More important than the directory tree is the **layering**. From bottom to top there are four layers:

Layer	Directories	Responsibility	Depends on
Platform / utility	`shared/`, `tool/`	Type aliases, file system, macros, delegates, image and model IO	Nothing (the base)
Graphics	`engine/vulkaninitialize/`, `graphics/`, `model/`	Turns Vulkan into resource objects plus one frame of drawing	Utility layer
Object	`scene/`, `reflection/`	Scene graph, components, metadata	Graphics layer (MeshComponent submits draws)

Layer	Directories	Responsibility	Depends on
Application / presentation	`SuzzaApplication`, `editor/`, `surface/`	Builds the scene, draws the UI, runs the loop	All of the above

Table 3-1 The four layers

One boundary matters most: `Engine` is only responsible for running (loop, window, Vulkan); `SuzzaApplication` is the game content. The engine never news a concrete Actor. Draw that line clearly and you always know where new code belongs.

3.3 Module responsibilities at a glance

Module	Key classes	One sentence to remember
Engine skeleton	Engine, MainWindow, BaseObject	Engine is the only singleton and the main scheduler; MainWindow owns window and input
Scene objects	Scene, Actor, Component	Scene owns actors, actors own components, parent actors own child actors
Render mainline	Render, BufferArea	Render sequences a frame, BufferArea records the actual draw commands
Vulkan resources	VulkanInitialize, SwapChain, RenderPass, Pipeline, Descriptor, SyncObject	One class per Vulkan concept, all owned by VulkanResourceManager
Resource management	ModelResourceManager, BindlessSlotsAllocator	Resources cached under random string ids; textures addressed by a 0-255 slot
Reflection	ClassInfo, FieldInfo, ReflectionSystem	Class to fields to getter/setter function objects; the editor edits through it
UI	IUIRender, ImGuiUIRender, EditorUI	surface abstracts the UI interface, editor implements the panels
Tools	ImageLoaderManager, ModelLoaderManager, Delegate	Manager plus scheduler plus implementation, easy to extend with new formats

Table 3-2 Module responsibilities

Chapter 4 Reading Route: Seven Stops

This is a route with **minimal prerequisites where every stop can be verified immediately**. Do not skip stops: each one builds on the concepts of the previous one.

Stop	Target files	Question this stop answers	Suggested time
1 Entry	`src/main.cpp`	Where does the program start? Which part is engine, which is game?	15 minutes
2 Engine skeleton	`src/engine/Engine.cpp`	What is the init order? What does the loop do each frame?	1 hour
3 Game layer	`src/SuzzaApplication.cpp`	How does a cube reach the screen?	1.5 hours
4 Object model	`src/scene/`	How do actors, components and the parent/child tree cooperate?	Half a day
5 Render mainline	MeshComponent + Render + BufferArea	How does a submit queue become a draw call?	Half a day
6 Vulkan resources	`src/engine/vulkaninitialize/`	Who creates and who destroys each Vulkan object?	1-2 days (can be skimmed)
7 Reflection and editor	`src/reflection/` + `editor/`	Why can the inspector edit arbitrary fields?	Half a day

Table 4-1 The seven reading stops

4.1 Stop 1: entry point (src/main.cpp)

Only 37 lines, but very dense:

```
SuzzaApplication* suzApp = new SuzzaApplication();
suz::Engine& engine = suz::Engine::GetEngineInstance();
constexpr int WIDTH = 1600, HEIGHT = 1200;
engine.SetApplication(suzApp);
if (!engine.Init()) { /* log the error and return */ }
engine.Run();
engine.Destroy();
```

Three things are enough for this stop:

1. `Engine` is a singleton, obtained through `GetEngineInstance()`.
2. Engine and game content are separated: the engine knows nothing about what lives in `SuzzaApplication`.
3. The lifetime trio `Init -> Run -> Destroy` owns everything created afterwards.

4.2 Stop 2: engine skeleton (src/engine/Engine.cpp)

The order inside `Engine::Init()` is the dependency order of the whole engine. Read it strictly in sequence:

Order	Action	Why it must happen here

Order	Action	Why it must happen here
1	`ReflectionSystem::RegisterAllReflectionClass()`	The editor and property system look classes up later, so the table must exist first
2	Create image and model loading schedulers	Needed by everything that loads assets afterwards
3	`new MainWindow(...)`	The Vulkan instance needs window extensions and a surface
4	`m_vkResourceManager.InitVulkan()`	Builds instance, device, swapchain, render pass, pipeline and command buffers in one go
5	`BindOnFrameBufferResizeCallback(...)`	Rebuilds size-dependent resources when the window is resized
6	`new Render()`	The renderer needs all Vulkan resources above to be ready
7	`m_editorUI->Init(...)` (EDITOR builds only)	ImGui needs a render pass and a command pool to hook into drawing
8	`m_suzzaApplication->Init()`	Game objects are created **last**, once every graphics resource exists

Table 4-2 Dependency order inside Engine::Init

`Engine::Run()` has a very small skeleton:

```
m_lastTime = steady_clock::now();
m_suzzaApplication->BeginPlay();
while (!m_mainWindow->WindowShouldClose()) {
    glfwPollEvents();
    m_editorUI->BeginFrame();           // EDITOR only
    float dt = now - m_lastTime;        // seconds
    m_suzzaApplication->Update(dt);     // logic
    m_render->Draw(dt);                 // rendering
    m_inputManager.SetMouseOldLocation(current);
}
vkDeviceWaitIdle(device);
```

BeginPlay runs only once. `BeginPlay` is called once, **outside** the loop; `Update` is called every frame **inside** it. This is the Unreal-style life cycle, and components have the same `BeginPlay / Update / Destroy` trio.

`Engine::Destroy()` is essentially the initialisation order reversed: application, editor, model resources, bindless slots, Vulkan resources, window. Reading teardown code is the best test of whether you really understood who depends on whom.

4.3 Stop 3: the game layer (src/SuzzaApplication.cpp)

141 lines, and the **best teaching sample in the project**. It does exactly one thing end to end: put three cubes on screen. Read it in four parts:

1. **Build geometry**: 24 vertices written by hand (6 faces x 4 vertices, each with position, colour and UV) and 36 indices (each face uses `0,1,2, 0,2,3`).

2. **Register resources**: `new Mesh(vertices, indices)` then `LoadMesh()` returns a string id; `new Material()` then `LoadMaterial()` returns a material id.
3. **Build the object tree**: `RootActor -> ChildA -> GrandChild`, each with a `MeshComponent` created through `CreateMesh(id)` and `CreateMaterial(id)`.
4. **Add the player**: `new PlayerController(scene)`, attach a `CameraComponent`, finally `SetActorLocation({0,0,0})`.

After this stop you should be able to answer: **'on this machine, what does my vertex data eventually turn into as a draw call?'** If you cannot, go back to section 5.4.

> **Your first exercise.** Lines 84-85 contain a commented-out `LoadModel(... "pony_cartoon/scene.gltf")`. Uncomment it and you switch from a hand-written cube to a real glTF model - the fastest way to understand `Scene::LoadModel`.

4.4 Stop 4: the object model (src/scene/)

This is **the architectural core**. Four files are enough; read them in the order `Component.h`, `Actor.h`, `Scene.h`, then a concrete component.

4.4.1 Component: the base of every capability

`Component` has only two important design decisions:

- **Pure virtual life cycle**: `BeginPlay()`, `Update(float)` and `Destroy()`, all implemented by subclasses.
- **Type ids**: a templated static counter hands every type a unique id, and a macro generates `GetTypeId()`.

```
template<typename T>
size_t GetStaticTypeId() {
    static size_t typeId = nextId++;
    return typeId;
}

#define COMPONENT(Comp) \
  static size_t GetComponentTypeId() { return GetStaticTypeId<Comp>(); } \
  size_t GetTypeId() const override { return GetComponentTypeId(); }
```

So `Actor::GetActorComponent<T>()` only has to walk the component list and compare `GetTypeId()`. Lookup by type works with no RTTI and no `dynamic_cast`.

4.4.2 Actor: entity and transform hierarchy

`Actor` holds several things at once; do not confuse them:

Member	Meaning
`m_components`	Components attached to this actor
`m_parentActor` / `m_childActors`	Parent/child tree, used for transform inheritance and recursive teardown
`m_scene`	Which scene it belongs to (an actor without a scene is never updated)

Member	Meaning
`m_status`	`Active` / `PendingDestroy` / `Destroyed`, the basis of deferred destruction
`m_location` / `m_rotation` / `m_scale`	Local transform; rotation is stored as `glm::quat` with an Euler-angle interface

Table 4-3 Key members of Actor

The transform is just a chain of matrix multiplications:

```
local = translate(m_location) * toMat4(m_rotation) * scale(m_scale)
world = parentWorld * local
model = world                          // handed to the shader as is
```

`SetActorWorldLocation()` deserves a separate look: it multiplies the target world position by the **inverse of the parent world matrix** to recover the local position. That is the standard way to convert between world and local space.

4.4.3 Scene: container and scheduler

`Scene` separates ownership from structure: a `vector<unique_ptr<Actor>>` owns every object while a `vector<Actor*>` keeps only the roots. Three behaviours must be understood:

- **Only roots are updated**: `Scene::Update` walks the root list only; children are updated recursively by their parent's `Update`. This guarantees parents update before children.
- **Deferred destruction**: `MarkedActorDestroy` only flips the status to `PendingDestroy`. The rest of the frame still runs, `Destroy()` is called at the end of the frame, and the element is erased from the vector on the next frame - so nothing is deleted while a loop is iterating.
- **BeginPlay happens in two passes**: first every actor registers its components, then roots get `BeginPlay` - so no component is used before it is attached.

Worth copying verbatim. `Scene::LoadModel()` is a complete 'file to scene' pipeline sample: walk `ModelData`, convert vertex layouts, build mesh and material, create actors, register them, attach components. When you add a new format later, copy the shape of this function.

4.5 Stop 5: the render mainline

This is the easiest stop to get lost in, but only **one queue** and **one recording pass** matter:

```
MeshComponent::BeginPlay()                // once, when resources are ready
  '- build ModelConstant{ modelMatrix, slotIndex }
     '- Render::SubmitMesh(mesh, constant)  // pushed into m_meshQueue

Render::Draw(dt)                           // every frame
  |- vkWaitForFences / vkAcquireNextImageKHR // wait, acquire
  |- BufferArea::RecordCommandBuffer(cmd,..) // record
  |    |- vkCmdBeginRenderPass
  |    |- for each mesh: BindVertexBuffers / BindIndexBuffer / DrawIndexed
  |    '- m_engineUIDelegate.Execute(cmd)    // ImGui draws here
  |- vkQueueSubmit
  '- vkQueuePresentKHR
```

The key insight: `MeshComponent` never draws directly. It submits 'what to draw with which matrix' into a queue, and `Render` records everything once per frame. Submit-then-draw-centrally is the norm in every engine: draw order and state sorting stay under central control.

4.5.1 The three lines in BufferArea that actually draw

`RecordCommandBuffer` is plain Vulkan code. As a beginner you only need to locate these calls:

Line (approx.)	Call	Effect
228	`vkCmdBeginRenderPass(...)`	Starts the render pass; everything after this lands in the framebuffer
231	`vkCmdBindPipeline(...)`	Binds the graphics pipeline (shaders plus fixed-function state)
255	`vkCmdBindVertexBuffers(...)`	Binds the vertex buffer of the current mesh
258	`vkCmdBindIndexBuffer(..., VK_INDEX_TYPE_UINT32)`	Binds the index buffer, 32-bit indices
267	`vkCmdDrawIndexed(indexCount, 1, 0, 0, 0)`	**the actual draw call**
272	`m_engineUIDelegate.Execute(cmd)`	ImGui commands are appended to the same render pass

Table 4-4 The calls that matter while recording

4.6 Stop 6: the Vulkan resource layer (can be skimmed)

`vulkaninitialize/` is organised as **one class per Vulkan concept**, so it can be used as a lookup table instead of read linearly:

Class	Vulkan concept	Notes
`VulkanInitialize`	Instance / surface / physical device / logical device / queues	Also handles validation layers and the `DescriptorIndexing` feature chain (the prerequisite for bindless)
`SwapChain`	Swapchain and image views	Rebuilt when the window is resized
`RenderPass`	Render pass (colour and depth attachments)	Defines the framebuffer formats
`Pipeline`	Graphics pipeline	Loads the spv files from `shaders/`
`Descriptor`	Descriptor pool, sets and layout	Camera UBO plus the bindless texture array
`BufferArea`	Vertex/index/UBO buffers, framebuffers, command pool and buffer	**The name is misleading**: it is really the whole set of swapchain-sized resources
`SyncObject`	Semaphores and fences	`MAX_FRAMES_IN_FLIGHT = 2`
`VulkanResourceManager`	Owner of everything above	`InitVulkan()` builds it all in dependency order

Table 4-5 Vulkan classes and the concepts they wrap

Do not sink into struct details. The right attitude for this stop: **build a map of which objects exist and who creates and destroys them; do not try to memorise every `VkCreateInfo` field.** Come back when you actually need to write code.

4.7 Stop 7: reflection and the editor

Understand this stop and you understand why the inspector can edit arbitrary fields. The chain has four steps:

1. **Annotate**: write `SUZZA_REFLECT_CLASS()` in the class and `SUZZA_PROPERTY(Editability::Editable)` on a field.
2. **Expand**: under clang the macro expands to `__attribute__((annotate("property:...")))`. Note that **under MSVC it expands to nothing** - it exists purely as a marker for the external tool.
3. **Generate metadata**: the external LLVM tool scans the sources and emits files such as `Actor.generated.h/.cpp` containing `ClassInfo{ name, path, fields }`, where each `FieldInfo` carries `getterFunc`, `setterFunc` and the field offset.
4. **Consume it**: `EditorUI::DrawInspectorPanel` walks `GetClassInfo().fields`, dispatches widgets by `typeName` (for example `glm::vec3` or `float`), calls the getter to read and the setter to write back.

So the magic is not at runtime but at **generation time**: the reflection data is generated code, not something discovered at runtime.

The trap that catches everyone. `SUZZA_REFLECT_CLASS()` builds an identifier out of `__LINE__` (for example `Actor_REFLECT_CLASS_30_REFLECT_BODY`). **Any line added or removed above that macro requires regenerating the reflection code**, otherwise the compiler reports a missing identifier. See section 5.6.

Chapter 5 Core Mechanisms

This chapter is a reference: jump to the section you need instead of reading it in order.

5.1 Engine lifetime

Phase	What happens	What not to do
`Init()`	Creates global resources in dependency order: window, Vulkan, renderer, editor, then the application's Init	Do not run the loop or submit draws inside Init
`Run()`	Loop: poll events, begin a UI frame, compute dt, update logic, render, snapshot input	Do not allocate global resources inside Run
`Destroy()`	Releases in reverse order and finishes with `vkDeviceWaitIdle` to ensure the GPU is idle	Do not forget ImGui or the slot allocator

Table 5-1 Responsibilities of the three phases

5.2 Object model: three rules to remember

- **Ownership**: Scene owns actors through `unique_ptr`; actors manage components with raw pointers plus `new` / `delete` (deleted one by one in `Destroy()`). The styles differ, so always note who is responsible for the delete.
- **Update order**: parent before child, and components before child actors. A parent transform is therefore already up to date when children run, so children see a consistent parent world matrix.
- **Deferred destruction**: any need to destroy while iterating goes through `MarkedActorDestroy` and is handled once at the end of the frame.

Why ValidPtr is everywhere. `Component` holds a `shared_ptr<bool> m_isAlive` and hands it out through `GetValidPtr()`, so after destruction outside code can still safely ask 'is it alive' and avoid dangling pointers. `Material`'s destructor checks it and logs a warning when `Destroy` was never called.

5.3 Resource management: random ids plus bindless slots

Mechanism	Problem it solves	Key API
`ModelResourceManager`	The same mesh or material is uploaded once; components refer to it by string id	`LoadMesh` / `LoadMaterial` return ids, `GetLoadedMesh(id)` looks them up
`BindlessSlotsAllocator`	Packs many textures into one descriptor array that the shader indexes	`AllocateBindlessSlots` / `MarkedTargetSlotsRelease` / `ReleaseSlots`

Table 5-2 The two resource caches

Why random strings instead of incrementing ids? Random ids avoid collisions with stale references after an unload and reload, and they can cross module boundaries without synchronising a counter. The cost is readability, so when debugging put a breakpoint on `LoadMesh` and read the returned value.

The texture slot limit comes from `MAX_SAMPLED_TEXTURE_IMAGES_SLOTS` (256) in `shared/macros/VkMacro.h`, and `MAX_FRAMES_IN_FLIGHT` (2) lives in the same file.

5.4 The render pipeline: from submit to present

Step	API	Error handling
Wait for the previous frame	`vkWaitForFences`	Blocks; no failure branch
Acquire an image	`vkAcquireNextImageKHR`	`VK_ERROR_OUT_OF_DATE_KHR` rebuilds the swapchain and skips the frame
Record	`vkResetCommandBuffer` + `RecordCommandBuffer`	Pure CPU-side command recording
Submit	`vkQueueSubmit` (waits imageAvailable, signals renderFinished)	Throws on failure
Present	`vkQueuePresentKHR`	`OUT_OF_DATE` / `SUBOPTIMAL` rebuild the swapchain; anything else throws
Advance the frame index	`m_currentFrame = (m_currentFrame + 1) % 2`	Matches the double-buffered frame resources

Table 5-3 The six steps of Render::Draw

Why two frames in flight? With one frame, the CPU must wait for the GPU to finish before preparing the next one; with two, the CPU can prepare frame N+1 while the GPU still draws frame N. That is also why the camera UBO, command buffers and semaphores are all arrays of length two.

Two indices, do not mix them. `m_currentFrame` is not the same thing as the swapchain's `imageIndex`: the first is the frame-resource index (always 2), the second is the image currently being drawn into (usually 3). Mixing them up produces synchronisation errors - the most common misreading of this code.

5.5 Vulkan object map: who creates, who destroys

Object	Created in	Destroyed in
`VkInstance` / debug messenger / surface	The `VulkanInitialize` init sequence	`VulkanInitialize::Destroy`
`VkDevice` / queues	`CreateLogicalDevice`	Same as above
`VkSwapchainKHR` + image views	`SwapChain::InitSwapChain`	`SwapChain::CleanUpSwapChain` plus rebuild
`VkRenderPass` + framebuffers	`RenderPass` / `BufferArea::CreateFrameBuffers`	Their own `Destroy` / `CleanupBufferArea`
`VkPipeline` / `VkPipelineLayout`	`Pipeline`	`Pipeline::Destroy`
`VkDescriptorPool/Set`	`Descriptor`	`Descriptor::Destroy`

Object	Created in	Destroyed in
`VkBuffer` + `VkDeviceMemory`	`BufferArea::CreateVertexBuffer` and friends	`CleanupBufferArea`
`VkImage` (textures / depth)	`Texture` / `CreateDepthResources`	Their own `Destroy`
Semaphores / fences	`SyncObject`	`SyncObject::Destroy`

Table 5-4 Ownership of Vulkan objects

5.6 Reflection: the full chain and three traps

5.6.1 Data structures

Type	Purpose	Key points
`FieldInfo`	Describes one field	Name, type name, owning class, **byte offset**, editability, getter and setter function objects
`ClassInfo`	Describes one class	Class name, source path, list of base classes, list of fields
`ReflectionSystem`	Global registry	`RegisterClassInfo` fills a map that is later searched by class name
`ContainerAccessor`	Reads and writes container fields	One uniform interface for types such as `Array`

Table 5-5 The four runtime reflection types

The field offset is computed with `SUZZA_OFFSET_OF(Actor::m_location)`: cast a null pointer to the class type and take the member address, which yields the offset. Getters and setters are compiler-generated lambdas, so runtime reflection costs almost nothing.

5.6.2 Three traps

1. **Line-number sensitivity**: the macro embeds `__LINE__` in an identifier (for example `Actor_REFLECT_CLASS_30_REFLECT_BODY`). Add or remove any line above it and the generated header no longer matches, producing an 'undefined identifier' error.
2. **External tool dependency**: the generator is a prebuilt executable that needs LLVM and `compile_commands.json`. A different machine, compiler or build directory may require reconfiguring.
3. **Absolute paths baked into generated files**: `classPath` in `Actor.generated.cpp` is a hard-coded string such as `D:/LjhUser/.../src/scene/Actor.h`. After moving the project that field is stale (today it only affects editor display, not compilation).

A workaround for beginners. If you only need to **add one field to an existing reflected class**, the cheapest path is to copy the shape of an existing `FieldInfo::CreateFieldInfo(...)` line, hand-write one line in the generated file and mirror the structure in the generated header; regenerate properly once the LLVM environment is available.

5.7 Editor, UI and input

- **UI abstraction**: `src/surface/UIRender.h` declares `IUIRender` and `ImGuiUIRender` implements it. Swapping UI libraries means swapping the implementation, not touching editor logic.
- **Editor wiring**: `EditorUI::Init` binds the draw function to `EngineUIRenderEventDelegate`; `BufferArea` calls `m_engineUIDelegate.Execute(cmd)` at the end of recording, so ImGui commands land in the same render pass.
- **Hierarchy**: walks scene roots and draws a tree with `TreeNodeEx`, plus click-to-select and a context menu that destroys actors.
- **Inspector**: walks `ClassInfo.fields` of the selected actor, dispatches by `typeName` (`glm::vec3` becomes three float drags, `float` becomes one), and calls the setter on change.
- **Input**: `InputManager` keeps two fixed arrays (256 keys, 16 mouse buttons); `MainWindow` is its friend and writes into them from GLFW callbacks. At the end of each frame the current mouse position is copied to the previous one so a delta can be computed.

Chapter 6 The Full Life Cycle of One Frame

Putting all the previous chapters together, one frame looks like this (the location is in brackets):

```
glfwPollEvents()                            (Engine::Run)
EditorUI::BeginFrame()                      (new ImGui frame)
deltaTime = now - lastTime

SuzzaApplication::Update(dt)                (game logic)
  '- Scene::Update
       '- roots only -> Actor::Update recurses into children
            '- each Component::Update
                 |- MovementComponent: read input -> change transform
                 '- MeshComponent:     (empty for now)

Render::Draw(dt)                            (rendering)
  |- vkWaitForFences
  |- vkAcquireNextImageKHR      -> rebuild swapchain if needed
  |- update camera UBO (current frame slot)
  |- BufferArea::RecordCommandBuffer
  |    |- BeginRenderPass
  |    |- walk m_meshQueue -> Bind + DrawIndexed
  |    |- ImGui commands (hierarchy / inspector)
  |    '- EndRenderPass
  |- vkQueueSubmit (wait imageAvailable, signal renderFinished)
  |- vkQueuePresentKHR
  '- m_currentFrame = (m_currentFrame + 1) % 2

InputManager: current -> old               (end-of-frame snapshot)
```

Figure 6-1 The timeline of one frame

Check yourself with four questions:

1. Why does `Scene::Update` only call `Update` on root actors? (Answer: parents before children, and no double updates.)
2. Why does a mesh submit itself in `BeginPlay` instead of every frame? (Answer: enqueueing is a one-off action when resources become ready; afterwards only the matrix changes.)
3. Why is `m_currentFrame` taken modulo 2? (Answer: double-buffered frame resources prevent the CPU from overwriting memory the GPU is still reading.)
4. Which resources must be rebuilt when the window is resized? (Answer: swapchain, image views, framebuffers and depth resources - all bound to the extent.)

Chapter 7 Hands-On Exercises

Changing it once beats reading it ten times. The tasks below are ordered by difficulty, 1 being the easiest, each with the files involved and an acceptance criterion.

Level	Task	Files involved	Done when
1	Change the window title and initial size	`src/main.cpp`	The title is what you chose, and resizing still draws correctly
1	Move and scale the cubes, watch the hierarchy follow	`SuzzaApplication.cpp`	Moving RootActor moves its children; moving ChildA moves GrandChild
2	Uncomment the model load and load `pony_cartoon`	`SuzzaApplication.cpp` lines 84-85	A full horse and cart appear with correct textures
2	Write a `RotateComponent` that spins around Y	`scene/components/`, copy `MovementComponent`	The cube rotates continuously with no input
3	Make the inspector show and edit a new `float` field	`reflection/`, `EditorUI.cpp`	The field shows in the panel and the picture follows your drag
3	Change the fragment shader so colour varies with UV	`simple_fragmentshader.frag`	After rebuilding, surfaces show a gradient or checker
4	Camera follow: camera position = target position + offset	`CameraComponent` + world transform	The view follows smoothly when the target moves
5	Add 'find an actor by name and destroy it' to Scene	`src/scene/Scene.*`	The named actor is removed without crashing (deferred destruction)

Table 7-1 Graded exercises

7.1 The standard way to add a component

Using `RotateComponent` as the example, the full procedure is below. This is also how you extend the engine in general:

1. Create `RotateComponent.h` under `src/scene/components/`, derive from `Component` and add the `COMPONENT(RotateComponent)` macro.
2. Implement `BeginPlay / Update / Destroy`; in `Update` call `m_actorOwner->SetActorRotation(...)`.
3. In `SuzzaApplication::Init`, call `actor->RegisterComponent<RotateComponent>()`.
4. Reconfigure and rebuild.

Do not forget to reconfigure. The cpp files under `src/scene/components/` are collected with `GLOB_RECURSE`, so **after adding a file you must re-run `cmake --preset`**, otherwise the linker reports undefined symbols.

Chapter 8 Common Pitfalls and Troubleshooting

8.1 At build time

Symptom	Cause	Fix
`compile_commands.json` not found	The Visual Studio generator was used, or configure never ran	Use the Ninja preset and run `cmake --preset` first
Reflection generation fails or code is stale	Missing generator executable or broken LLVM environment	Check the executable in `tool/ReflectGenerated/`; maintain generated files by hand if needed
`glslc` not found	Vulkan SDK not installed or not on PATH	Install the SDK and confirm `find_package(Vulkan)` succeeds
Garbled comments or warning C4819	Sources are GBK-encoded; the editor opens them as UTF-8	Set `files.encoding: gbk`; keep new files consistent with the repository
Undefined symbols after adding a cpp file	`GLOB` was not refreshed	Re-run `cmake --preset`
Strange incremental build errors	Stale build cache	Delete `out/build/x64-debug` and configure again

Table 8-1 Build-time problems

8.2 At run time

Symptom	Most likely cause
Window opens but everything is black	Validation-layer descriptor or pipeline errors; or a camera matrix pointing away
Model fails to load or file not found	`assets/` not copied next to the executable, or `GetAssetsFolder()` resolves elsewhere
Textures are all black or all white	Bindless slot not allocated or not released; or UV and sampler mismatch
Crash or corruption after resizing	Swapchain resources were not rebuilt for the new extent
ImGui panel is invisible	Draw callback not bound, or not recorded inside the render pass
Editing a field does nothing	Field not marked `SUZZA_PROPERTY`, or the edit did not trigger a redraw

Table 8-2 Run-time problems

8.3 Three known issues left in the repository

- `README_zh.md` still contains Git conflict markers (`<<<<<<< HEAD` and friends) around lines 14-18 and 50-57; skip those paragraphs.
- The generated reflection files hard-code the original author's absolute paths (see 5.6.2).
- `complieShader.bat` hard-codes a local Vulkan SDK path; use it only as a reference.

Chapter 9 Glossary

Term	Meaning in this project
Actor	An entity in the scene: name, transform, component list, optional parent and children
Component	A capability attached to an actor implementing `BeginPlay / Update / Destroy`
Scene	The container and world of actors; handles update, destruction and model loading
BeginPlay	Life-cycle hook run when an object enters the running state, called exactly once
Deferred destruction	Do not delete while iterating: set `PendingDestroy` and destroy at the end of the frame
bindless	Many textures placed in one descriptor array and indexed from the shader
slot	Index into that array; this project allows up to 256
frames in flight	How many frames the CPU may prepare ahead of the GPU; 2 here
swapchain	The set of images used for presentation; rebuilt on resize
render pass	The attachment and subpass description one drawing operation depends on
reflection	Querying class field information at runtime; here produced by code generation
ClassInfo / FieldInfo	The runtime reflection structures describing a class and a field
ModelConstant	Per-object constants sent to the shader: model matrix plus texture slot
Pipeline	The pipeline state object that fixes shaders and fixed-function state

Table 9-1 Terms used in this document

Annex A File Quick Reference

Main files grouped by directory, one line of responsibility each, for those moments when you know what a feature does but not which file implements it.

A.1 Engine core (src/engine/)

File	Responsibility
`Engine.h/.cpp`	Singleton and main scheduler: Init / Run / Destroy; owns window, Vulkan, renderer, editor
`MainWindow.h/.cpp`	GLFW window, input callbacks, resize delegate, surface creation
`BaseObject.h`	Common base of all resources: `Destroy()` plus `IsAlive / GetValidPtr`
`input/InputManager.h`	Key and mouse state arrays, mouse position snapshot
`editor/EditorUI.h/.cpp`	Hierarchy tree plus inspector property table
`graphics/Texture.h/.cpp`	VkImage + image view + memory allocation + slot update
`model/Mesh.h/.cpp`	Vertex and index buffers plus the vertex layout description
`model/Material.h/.cpp`	A material is one texture plus one bindless slot
`model/Render.h/.cpp`	Frame sequencing: acquire, record, submit, present; owns the mesh submit queue
`model/ModelResourceManager.h`	Id cache and reverse lookup for meshes and materials
`model/BindlessSlotsAllocator.h`	Texture slot allocation and deferred release

A.2 Vulkan layer (src/engine/vulkaninitialize/)

File	Responsibility
`VulkanInitialize.h/.cpp`	Instance, surface, physical and logical device, queues, validation layers, feature chain
`SwapChain.h/.cpp`	Swapchain, image views, rebuild
`RenderPass.h/.cpp`	Render pass with colour and depth attachments
`Pipeline.h/.cpp`	Graphics pipeline and layout
`Descriptor.h/.cpp`	Descriptor pool, layout and sets (camera plus bindless textures)
`BufferArea.h/.cpp`	Vertex/index/UBO buffers, framebuffers, command pool and buffers, command recording
`SyncObject.h/.cpp`	Semaphores and fences
`VulkanResourceManager.h/.cpp`	Owns all of the above and defines the init order

A.3 Scene and reflection (src/scene/, src/reflection/)

File	Responsibility
`scene/Scene.h/.cpp`	Actor registry, root update, deferred destruction, model loading pipeline
`scene/Actor.h/.cpp`	Entity: transform, component list, parent/child tree, lookup by type

File	Responsibility
scene/Component.h	Component base class, type id mechanism, alive flag
scene/components/MeshComponent.*	Submits mesh, material and model matrix to Render
scene/components/CameraComponent.*	View and projection matrices, FOV, clip planes
scene/components/MovementComponent.*	Mouse look plus WASD movement
scene/actor/PlayerController.*	Actor subclass carrying a MovementComponent
reflection/ReflectionInfo.h/.cpp	ClassInfo, FieldInfo, ReflectionSystem
reflection/ContainerAccessor.h	Uniform read/write interface for container fields

A.4 Shared code and tools (shared/, tool/)

File	Responsibility
shared/SuzzaTypes.h	Type aliases such as suz::uint32
shared/file/FileSystem.cpp	Resolves the executable directory, source directory, shaders and assets
shared/macros/VkMacro.h	Constants such as MAX_FRAMES_IN_FLIGHT and the texture slot limit
shared/macros/ReflectMacros.h	All reflection-related macros
tool/broadcast/	Unicast and multicast delegates (signals and slots) used for resize and UI draw callbacks
tool/imageloader/	Image loading: manager, scheduler, stb_image implementation
tool/modelloader/	Model loading: manager, scheduler, cgltf implementation
tool/ReflectGenerated/	The reflection generator executable and the generated metadata code
src/core/debug/SuzzaLog.h	Logging macros, compiled out in production builds

Annex B Key API Quick Reference

B.1 Engine and scene

```
suz::Engine& e = suz::Engine::GetEngineInstance();
e.SetApplication(app);  e.Init();  e.Run();  e.Destroy();

auto* window   = suz::Engine::GetEngineInstance().GetMainWindow();
auto* render   = suz::Engine::GetEngineInstance().GetRender();
auto& modelMgr = suz::Engine::GetModelResourceManager();

scene->RegisterActor(actor);
scene->RegisterActor(child, parent, /*bRegisterComponents=*/false);
scene->MarkedActorDestroy(actor);      // deferred destruction
```

B.2 Actors and components

```
actor->RegisterComponent<MeshComponent>();
auto* mesh = actor->GetActorComponent<MeshComponent>();
auto  all  = actor->GetAllActorComponents();

actor->SetActorLocation(glm::vec3(1, 0, 0));          // local space
actor->SetActorWorldLocation(glm::vec3(3, 1.2f, 0));  // world space
glm::mat4 m = actor->GetActorWorldTransform();
actor->GetActorForwardVector();   // Right / Up exist too
```

B.3 Submitting renderables

```
auto meshId = modelMgr.LoadMesh(vertices, indices);   // or LoadMesh(Mesh*)
auto matId  = modelMgr.LoadMaterial(new suz::Material(texturePath));

auto* mc = new suz::MeshComponent();
mc->CreateMesh(meshId);
mc->CreateMaterial(matId);
actor->RegisterComponent<suz::MeshComponent>();
```

B.4 Reflection annotations

```
class MyThing : public suz::Actor {
  SUZZA_REFLECT_CLASS()                // line sensitive: __LINE__ is in the name
public:
  SUZZA_PROPERTY(suz::Editability::Editable)
  float m_myFloat = 1.0f;
};
```

- End of document -